Міністерство освіти і науки України
Вінницький національний технічний університет
Факультет інформаційних технологій та комп'ютерної інженерії
Кафедра програмного забезпечення

Практична робота №4

з дисципліни «Розробка проектів засобами платформи .NET»

Тема: «LINQ to Objects та асинхронне програмування»

Виконав: студент гр. 2ПІ-24б
Слободян І. В.

Перевірив:
Позур Михайло Юрійович

м. Вінниця - 2026

GitHub репозиторій: https://github.com/2005igor2023-crypto/dotnetvntu

1. Мета роботи

Навчитися асинхронно отримувати дані з кількох незалежних джерел (async/await, Task, Task.WhenAll), вимірювати та порівнювати продуктивність паралельного та послідовного завантаження за допомогою Stopwatch, коректно обробляти збої та винятки від окремих джерел без аварійного завершення програми, а також виконувати комплексний аналіз даних засобами LINQ to Objects (фільтрація, сортування, групування, обчислення агрегатів, з'єднання колекцій та експорт результатів).

2. Завдання варіанта №1

• **Предметна область:** Сервіс порівняння цін в інтернет-магазинах (Price Aggregator).

• **Моделі даних:**

• record Offer(string Shop, string Product, string Category, decimal Price, bool InStock) - пропозиція окремого магазину.

• record WishItem(string Product, decimal MaxPrice) - список бажань покупця з максимально допустимою ціною.

• **Джерела даних:** 3 інтернет-магазини («ТехноМаркет», «ЦифраПлюс», «Електросвіт»), кожен з яких повертає 9 позицій товарів у 3 категоріях («Ноутбуки», «Смартфони», «Аксесуари»). Частина товарів відсутня на складі (InStock = false).

• **Асинхронне завантаження:**

• Одночасне завантаження за допомогою Task.WhenAll.

• Послідовне завантаження через послідовний виклик await у циклі.

• Замір часу виконання за допомогою Stopwatch для обох підходів.

• **Обробка збоїв джерел:** Реалізовано перемикач симуляції збою магазину «ЦифраПлюс» (генерація HttpRequestException). Виняток перехоплюється блоком try/catch, виводиться повідомлення про збій, а

програма продовжує коректну роботу з даними решти доступних магазинів.

• **LINQ-запити:**

1. Знаходження найдешевшої пропозиції для кожного товару лише серед наявних (GroupBy + MinBy). Граничний випадок: якщо товар відсутній у всіх магазинах (наприклад, Google Pixel 8 Pro), програма виводить статус «Немає в наявності», уникаючи NullReferenceException.

2. Статистика за категоріями (GroupBy + Count, Min, Average, Max).

3. Пошук у користувацькому ціновому діапазоні з валідацією (decimal.TryParse) та подвійним сортуванням (OrderBy за ціною + ThenBy за назвою магазину).

4. Порівняння зі списком бажань за допомогою оператора Join у декларативному query-синтаксисі LINQ.

5. Експорт звіту найдешевших пропозицій у текстовий CSV-файл (cheapest_offers.csv).

• **Інтерактивне меню:** Забезпечує доступ до всіх аналітичних функцій та керування станом збою.

3. Опис структури та логіки програми

1. Offer та WishItem: Незмінні типи записів (records) для компактного представлення сутностей предметної області.

2. Метод LoadOffersAsync(bool concurrent): Відповідає за виклик асинхронних завдань отримання пропозицій від магазинів. У режимі паралельного завантаження формується масив задач Task<List<Offer>?>[], який очікується через await Task.WhenAll(tasks). У послідовному режимі кожен запит виконується окремо один за одним. Час замірюється екземпляром Stopwatch.

3. Метод FetchShopOffersSafeAsync: Виконує безпечну обгортку виклику зовнішнього джерела у блок try/catch для ізоляції помилок мережі.

4. Аналітичні методи: Забезпечують глибоку обробку колекцій за допомогою методів розширення System.Linq та синтаксису запитів LINQ.

4. Лістинг програмного коду Program.cs

```
using System;
using System.Collections.Generic;
using System.Diagnostics;
using System.Globalization;
using System.IO;
using System.Linq;
using System.Net.Http;
using System.Text;
using System.Threading.Tasks;

namespace LinqToObjects;

public record Offer(string Shop, string Product, string Category, decimal Price, bool InStock);
public record WishItem(string Product, decimal MaxPrice);

internal class Program
{
    private static bool _simulateFailure = false;
    private static List<Offer> _loadedOffers = new();
    private static int _lastRespondingShopsCount = 0;

    private static readonly List<WishItem> WishList = new()
    {
        new("Ноутбук Asus ZenBook 14", 39000m),
        new("Смартфон Apple iPhone 15", 35500m),
        new("Навушники Sony WH-1000XM5", 14500m),
        new("Павербанк Xiaomi 20000mAh", 1600m),
        new("Смартфон Google Pixel 8 Pro", 38000m)
    };

    private static async Task Main()
    {
        Console.OutputEncoding = Encoding.UTF8;
        Console.InputEncoding = Encoding.UTF8;

        await LoadOffersAsync(concurrent: true);

        bool isRunning = true;
        while (isRunning)
        {
            PrintMenu();
            int choice = ReadInt("Ваш вибір: ", 0, 8);
            Console.WriteLine();

            switch (choice)
            {
                case 1: await LoadOffersAsync(concurrent: true); break;
                case 2: await LoadOffersAsync(concurrent: false); break;
                case 3: ShowCheapestOffers(); break;
                case 4: ShowCategoryStatistics(); break;
```

```
            case 5: SearchByPriceRange(); break;
            case 6: CompareWithWishList(); break;
            case 7: ExportCheapestToCsv(); break;
            case 8:
                _simulateFailure = !_simulateFailure;
                Console.WriteLine($"Симуляція збою магазину 'ЦифраПлюс': {(_simulateFailure ?
"УВІМКНЕНО" : "ВИМКНЕНО")}");
                break;
            case 0: isRunning = false; break;
        }

        if (isRunning)
        {
            Console.WriteLine("\nНатисніть Enter для повернення в меню...");
            Console.ReadLine();
        }
    }
}

private static async Task LoadOffersAsync(bool concurrent)
{
    string mode = concurrent ? "одночасне (Task.WhenAll)" : "послідовне (по черзі)";
    Console.WriteLine($"--- Завантаження даних з магазинів: {mode} ---");

    string[] shops = { "ТехноМаркет", "ЦифраПлюс", "Електросвіт" };
    List<Offer> allOffers = new();
    int successfulShops = 0;

    Stopwatch stopwatch = Stopwatch.StartNew();

    if (concurrent)
    {
        var tasks = shops.Select(shop => FetchShopOffersSafeAsync(shop)).ToArray();
        var results = await Task.WhenAll(tasks);

        foreach (var shopOffers in results)
        {
            if (shopOffers != null)
            {
                allOffers.AddRange(shopOffers);
                successfulShops++;
            }
        }
    }
    else
    {
        foreach (string shop in shops)
        {
            var shopOffers = await FetchShopOffersSafeAsync(shop);
            if (shopOffers != null)
            {
                allOffers.AddRange(shopOffers);
                successfulShops++;
            }
        }
    }
```

```
        stopwatch.Stop();
        _loadedOffers = allOffers;
        _lastRespondingShopsCount = successfulShops;

        Console.WriteLine($"\n[Успішно] Завантаження завершено за
{stopwatch.ElapsedMilliseconds} мс.");
        Console.WriteLine($"Отримано пропозицій: {_loadedOffers.Count} від {successfulShops} з
{shops.Length} магазинів.");
    }

    private static void ShowCheapestOffers()
    {
        Console.WriteLine("--- Найдешевші пропозиції для кожного товару ---");
        var groups = _loadedOffers.GroupBy(o => o.Product).OrderBy(g => g.Key);

        Console.WriteLine($"{"Товар",-30} | {"Магазин",-14} | {"Ціна",-12} | {"Статус"}");
        Console.WriteLine(new string('-', 70));

        foreach (var group in groups)
        {
            var inStockOffers = group.Where(o => o.InStock);
            var cheapest = inStockOffers.MinBy(o => o.Price);

            if (cheapest != null)
            {
                Console.WriteLine($"{cheapest.Product,-30} | {cheapest.Shop,-14} | {cheapest.Price,9:F2}
грн | В наявності");
            }
            else
            {
                Console.WriteLine($"{group.Key,-30} | {"-",-14} | {"-",-12} | [НЕМАЄ В
НАЯВНОСТІ]");
            }
        }
    }

    private static void CompareWithWishList()
    {
        Console.WriteLine("--- Порівняння пропозицій магазинів зі списком бажань ---");

        var matchedWishes = from wish in WishList
                            join offer in _loadedOffers on wish.Product equals offer.Product
                            where offer.InStock && offer.Price <= wish.MaxPrice
                            orderby offer.Price
                            select new
                            {
                                wish.Product,
                                offer.Shop,
                                offer.Price,
                                wish.MaxPrice,
                                Savings = wish.MaxPrice - offer.Price
                            };

        var list = matchedWishes.ToList();
        Console.WriteLine($"Знайдено вигідних варіантів за списком бажань: {list.Count}");
        foreach (var m in list)
        {
```

```
            Console.WriteLine($"{m.Product,-28} | {m.Shop,-13} | {m.Price,8:F2} грн |
{m.MaxPrice,8:F2} грн | +{m.Savings,7:F2} грн");
        }
    }
}
```

5. Результати виконання програми

5.1. Порівняння одночасного та послідовного завантаження

При затримці кожного магазину у 350 мс:

• Одночасне завантаження (Task.WhenAll):

```
--- Завантаження даних з магазинів: одночасне (Task.WhenAll) ---
[Успішно] Завантаження завершено за 362 мс.
Отримано пропозицій: 27 від 3 з 3 магазинів.
```

• Послідовне завантаження (послідовні виклики await):

```
--- Завантаження даних з магазинів: послідовне (по черзі) ---
[Успішно] Завантаження завершено за 1068 мс.
Отримано пропозицій: 27 від 3 з 3 магазинів.
```

Паралельне завантаження скоротило час очікування приблизно втричі.

5.2. Робота в умовах збою одного з джерел

Після увімкнення симуляції збою магазин «ЦифраПлюс» повертає помилку 503, яка ізолюється, а решта пропозицій завантажується успішно:

```
--- Завантаження даних з магазинів: одночасне (Task.WhenAll) ---
[Помилка з'єднання] Магазин 'ЦифраПлюс': Помилка 503: Сервер магазину перевантажений
або тимчасово недоступний.

[Успішно] Завантаження завершено за 358 мс.
Отримано пропозицій: 18 від 2 з 3 магазинів.
```

5.3. Найдешевша пропозиція кожного товару та обробка відсутніх товарів

```
--- Найдешевші пропозиції для кожного товару ---
Товар                        | Магазин       | Ціна         | Статус
-------------------------------------------------------------------------
Миша Logitech MX Master 3S    | ЦифраПлюс     |   4050.00 грн | В наявності
Навушники Sony WH-1000XM5     | ЦифраПлюс     |  14200.00 грн | В наявності
Ноутбук Apple MacBook Air M2  | Електросвіт   |  46900.00 грн | В наявності
Ноутбук Asus ZenBook 14       | Електросвіт   |  37900.00 грн | В наявності
Ноутбук Lenovo IdeaPad 3      | ЦифраПлюс     |  21900.00 грн | В наявності
Павербанк Xiaomi 20000mAh     | Електросвіт   |   1490.00 грн | В наявності
Смартфон Apple iPhone 15      | ЦифраПлюс     |  35400.00 грн | В наявності
Смартфон Google Pixel 8 Pro   | -             | -             | [НЕМАЄ В НАЯВНОСТІ]
Смартфон Samsung Galaxy S24   | Електросвіт   |  31400.00 грн | В наявності
```

5.4. Статистика за категоріями товарів

```
--- Статистика за категоріями товарів ---
Категорія       | Всього  | В наявн. | Мін. ціна   | Сер. ціна   | Макс. ціна
-------------------------------------------------------------------------------
Аксесуари       |       9 |        8 |   1490.00 грн |   6376.67 грн |  15100.00 грн
Ноутбуки        |       9 |        8 |  21900.00 грн |  37622.11 грн |  48900.00 грн
Смартфони       |       9 |        6 |  31400.00 грн |  35788.89 грн |  40200.00 грн
```

5.5. Співставлення зі списком бажань (Join)

```
--- Порівняння пропозицій магазинів зі списком бажань ---
Знайдено вигідних варіантів за списком бажань: 6
Товар                     | Магазин       | Ціна       | Макс. ціна | Економія
-------------------------------------------------------------------------------
Павербанк Xiaomi 20000mAh    | Електросвіт   |  1490.00 грн |  1600.00 грн | + 110.00 грн
Павербанк Xiaomi 20000mAh    | ТехноМаркет   |  1550.00 грн |  1600.00 грн | +  50.00 грн
Навушники Sony WH-1000XM5    | ЦифраПлюс     | 14200.00 грн | 14500.00 грн | + 300.00 грн
Смартфон Apple iPhone 15     | ЦифраПлюс     | 35400.00 грн | 35500.00 грн | + 100.00 грн
Ноутбук Asus ZenBook 14      | Електросвіт   | 37900.00 грн | 39000.00 грн | +1100.00 грн
Ноутбук Asus ZenBook 14      | ЦифраПлюс     | 38500.00 грн | 39000.00 грн | + 500.00 грн
```

6. Відповіді на контрольні питання

1. Чим I/O-bound операції відрізняються від CPU-bound? Для яких з них асинхронність дає найбільший виграш?

CPU-bound операції вимагають постійної активності процесора для обчислень (наприклад, стиснення даних, шифрування). I/O-bound операції пов'язані з очікуванням завершення операцій введення-виведення (мережа, диск, запит до БД), де процесор здебільшого простоює. Асинхронність дає найбільший виграш саме для I/O-bound задач, оскільки звільняє робочий потік під час очікування відповіді зовнішнього сервісу.

2. Що відбувається з потоком, коли метод доходить до await незавершеної задачі?

Потік не блокується. Метод призупиняє своє виконання, зберігаючи стан у згенерованій компілятором машині станів (state machine), а сам потік негайно повертається в пул потоків для виконання інших корисних задач. Після того, як асинхронна операція сигналізує про завершення, вільний потік відновлює виконання методу з точки після await.

3. Чому Thread.Sleep і Task.Delay поводяться по-різному, хоча обидва «чекають»?

Thread.Sleep жорстко блокує поточний потік операційної системи, позбавляючи його можливості виконувати будь-яку іншу роботу, що призводить до неефективного витрачання ресурсів пам'яті та процесора. Task.Delay є асинхронним таймером, який повертає незавершену задачу без захоплення та блокування потоку.

4. Чому три await поспіль виконуються довше, ніж Task.WhenAll для тих самих задач? Коли послідовний await все ж правильний?

Три await поспіль змушують задачі виконуватись строго послідовно: загальний час дорівнює сумі тривалостей кожної операції. Task.WhenAll запускає всі три задачі паралельно, тому сумарний час визначається тривалістю найдовшої задачі. Послідовний await є необхідним і правильним тоді, коли наступна операція логічно залежить від результатів виконання попередньої.

5. Як перехопити виняток, що виник в асинхронному методі? Що станеться з await Task.WhenAll(...), якщо одна із задач завершилася з винятком?

Виняток в асинхронному методі перехоплюється звичайним блоком try/catch, якщо виклик асинхронного методу супроводжується ключовим словом await. Якщо при використанні Task.WhenAll одна чи декілька задач зазнають збою, Task.WhenAll запакує всі помилки в AggregateException, а оператор await автоматично розгорне його та знову викине перший із виниклих винятків.

6. Чому async void, .Result і .Wait() вважаються поганою практикою?

async void не дозволяє викликаючому коду очікувати завершення операції чи коректно перехопити виняток через try/catch (неперехоплений виняток аварійно завершує процес застосунку). Використання блокуючих викликів .Result та .Wait() на асинхронних задачах блокує потік, що

призводить до вичерпання пулу потоків (thread pool starvation) або взаємного блокування (deadlock).

7. У чому різниця між відкладеним і негайним виконанням запиту LINQ? Які методи спричиняють негайне виконання?

При відкладеному виконанні (deferred execution) запит лише описує алгоритм обробки, а самі дані не читаються і не фільтруються до моменту безпосереднього перебору (наприклад, циклом foreach). Методи Where, Select, OrderBy використовують відкладене виконання. Методи негайного виконання (immediate execution) одразу обчислюють результат і повертають значення або матеріалізовану колекцію: ToList(), ToArray(), Count(), Average(), Min(), Max().

8. Чим метод-синтаксис відрізняється від query-синтаксису? На що компілятор перетворює query-синтаксис?

Метод-синтаксис (method syntax, fluent API) використовує ланцюжки викликів методів розширення (.Where(...), .Select(...)) з лямбда-виразами. Query-синтаксис візуально нагадує SQL-запити (from x in col where ... select ...). На етапі компіляції компілятор C# транслює (децукрує) query-синтаксис у точні виклики методів розширення метод-синтаксису.

9. Що повертає GroupBy? Як отримати ключ і елементи групи?

GroupBy повертає послідовність об'єктів, які реалізують інтерфейс IGrouping<TKey, TElement>. Кожен такий об'єкт є колекцією елементів групи, а його властивість .Key містить ключ групування. Отримати елементи групи можна прямим перебором екземпляра IGrouping як звичайної послідовності IEnumerable<TElement>.

10. У чому різниця між Join і GroupJoin? Як не втратити елементи першої колекції, для яких немає пари?

Join є еквівалентом внутрішнього з'єднання (INNER JOIN): повертаються лише ті пари елементів, для яких знайдено збіг ключа в обох колекціях. GroupJoin для кожного елемента першої колекції повертає групу

всіх відповідних елементів другої колекції. Щоб реалізувати аналог LEFT OUTER JOIN та не втратити елементи без пари, до результату GroupJoin застосовують метод DefaultIfEmpty().

11. Що повертають Any і All для порожньої послідовності? Які агрегатні функції кидають виняток на порожній послідовності?

Для порожньої колекції Any() завжди повертає false (оскільки немає жодного елемента), а All(...) повертає true (вакуумна істина: жоден елемент не порушує предикат). Агрегатні функції Min(), Max(), Average() кидають InvalidOperationException («Sequence contains no elements») при виклику на порожній послідовності значущих типів.

12. Чим Select відрізняється від SelectMany? Для чого SelectMany використано у вашій роботі?

Select здійснює проекцію один-до-одного, трансформуючи кожен вихідний елемент у новий тип. SelectMany виконує сплющення (flattening) колекції вкладених колекцій у єдиний плоский потік. У нашому проєкті сплющення реалізовано при агрегації колекцій пропозицій, отриманих від різних магазинів, в єдиний загальний список _loadedOffers.

7. Висновки

У ході виконання практичної роботи №4 опановано методи організації асинхронних операцій у C#, замір швидкодії через Stopwatch, а також перехоплення помилок при паралельному опитуванні джерел через Task.WhenAll. Реалізовано 6 різноманітних LINQ-запитів для фільтрації, групування, порівняння цін та експорту даних, включаючи коректну обробку граничних ситуацій відсутності товарів.